



Lecture 4 — The Elephant-Goldfish Model

AI-Assisted Software Development and Intelligent Applications

Today's Agenda

1. Why AI coding needs a process, not just a prompt
2. The Elephant-Goldfish Model — four phases
3. The context hierarchy
4. **Activity** — begin the Book Review arc

Pre-Class Checklist — Start the Quiz

Did you complete the reading and the exercise?

- Read: *Elephants, Goldfish and the New Golden Age of Software Engineering* (Rensin, 2026)
- Generated a **context hierarchy** (nested `CLAUDE.md` files) for the Book Review starter, leaves to root

If you didn't finish it, start now — you'll need it shortly.

Did anyone think to create a **skill** to build the context hierarchy?

Two Failure Modes, No Process

An assistant with no process is like an **elephant with amnesia**: huge capability, no memory between sessions, no instinct for when it's actually done.

- **Over-ambition** — attempts everything at once, runs out of context mid-build
- **Premature victory** — declares "done" when it isn't

Elephant + Goldfish

	The problem	The fix
Elephant	Forgets everything between sessions	Build durable, external memory
Goldfish	A fresh context remembers nothing	Use that as a test

A fresh session is a goldfish. That's not a bug — it's the sharpest tool for checking whether your design doc actually stands alone.

Four Phases

Prerequisite — context hierarchy: build once, refresh as the codebase changes.

1. **Growing the Elephant** — load context, discuss design, no code
2. **Teaching the Elephant** — write the design doc
3. **Goldfish Protocol** — test the doc, context-free
4. **Implementation** — build it, then tear it apart

Today: Phase 1 in class, Phases 2–4 as homework, on one small feature.

Claude Code Permission Modes

Sessions **start in Auto**. Cycle with **Shift+Tab**:

`Auto` → `Manual` → `Accept Edits` → `Plan` → back to `Auto`

Mode	What runs without asking
Auto (default)	Everything — a background safety classifier reviews actions instead of you
Manual	Nothing — every edit and command needs your OK
Accept Edits	File edits apply automatically; you check <code>git diff</code> after
Plan	Reads only — Claude researches and proposes a plan; <i>no edits until you approve it</i>

Modes Across the Four Phases

- Auto still lets Claude write code — a classifier reviews it, but the No Code Rule needs more than that. For **Phases 1–3**, explicitly switch to **Plan Mode** (`Shift+Tab`, or prefix a prompt with `/plan`) — only there is Claude physically blocked from editing until you approve its plan.
- **Approving the plan** (start of Phase 4) — choose Manual to review each change, Accept edits to go faster and check the diff after, or drop back to Auto and let the classifier decide.
- `Ctrl+G` while planning opens the proposed plan in your editor — an easy way to turn it into your first draft of `SPEC.md` directly.

Turn Phases Into Skills

A `.claude/skills/<name>/SKILL.md` packages a procedure you repeat every time — Claude invokes it automatically when relevant, or you call it directly with `/name`.

- **Context hierarchy** — same leaves-to-root steps, every refresh
- **Sycophant challenge** — the same three pushback questions, every Phase 1
- **Goldfish Protocol** — fresh session, feed the spec, same three checks
- **Mean code review** — the same "tear this apart" prompt, every Phase 4

Encode what you'd otherwise have to remember to do the same way each time.

Phase 1: Growing the Elephant

Context loading — Claude needs to understand the codebase first. That's what your context hierarchy (nested `CLAUDE.md` files, auto-loaded as Claude explores) is for.

The No Code Rule — no implementation in this phase. Deciding what to build stays separate from building it.

First draft proposal — ask Claude to describe its approach in prose. A starting point, not something to accept.

The Sycophant Challenge

Assistants are biased toward agreeing with their own first proposal.
Push back, explicitly:

- *"What could go wrong with this?"*
- *"What edge cases haven't we covered?"*
- *"Is there a simpler way?"*

Not about being difficult — it's a check against a documented bias.

Buzzer Checkpoint

**Launch Question 6 – discuss answers, then
continue**

The Context Hierarchy

Problem: dumping a whole codebase into context is expensive and unfocused.

Technique: recursive summarization, leaves to root — the pre-class reading's "README hierarchy" idea, built as nested `CLAUDE.md` files instead of `README.md` files, so Claude Code loads them automatically and they never compete with every project's human-facing `README.md`.

1. Summarize the deepest source directories first
2. Verify each summary before moving up
3. One level up, summarize the summaries below it — not raw source again
4. Repeat to the root

Staying synced: a `.claude/rules/` file can require updates whenever a directory's source changes.

Why It Scales

A context hierarchy is the same compression trick you already use as a human — you read docs, not a library's whole source, before using it.

For Project 2 (~13,000 lines across three frontends and a backend), this is the difference between orienting in seconds and burning your whole context window just finding things.

Phase 2: Teaching the Elephant

The design doc — `SPEC.md` in this course — built iteratively:

- **Problem** — what are we solving, in a sentence or two?
- **Technical Plan** — the agreed approach
- **Alternatives** — what else was considered, and why it lost
- **Detailed Implementation** — specific enough to name what changes
- **Keep it:** one file per feature in `docs/plans/`, not a single `SPEC.md` overwritten each time

Test: could someone with *zero* context implement this from the doc alone?

Phase 3: The Goldfish Protocol

Feed the design doc + **context hierarchy** to a brand-new session (`/clear`) — never the discussion that produced them. "Context-free" means free of that conversation, not the docs any real implementer would also have.

- **Comprehension test** — does it explain the feature back correctly?
- **Critic review** — what holes does it find?
- **Readiness check** — would it know exactly what to build?

Any gap goes back into the spec — revised *before* implementation starts. Stay in **Plan Mode** — you're testing comprehension, not ready to edit yet.

Buzzer Checkpoint

**Launch Question 7 – discuss answers, then
continue**

Phase 4: Implementation

Only now does code get written — against the goldfish-tested spec. Approve the plan (or exit Plan Mode with `Shift+Tab`) and pick Manual or Accept Edits:

- Point Claude at the finished `SPEC.md`, not a re-explanation from memory - Implement to the spec, verify against tests as you go

Mean code review — once "done," ask Claude to tear its own implementation apart. A different posture than generating: critical review catches what generation alone doesn't.

EGM on Every Project From Here

Phase	Book Review arc	Projects 2-4 / Final
Context	Provided starting structure	Bottom-up context hierarchy you write
Elephant → Goldfish	Design + SPEC.md, fresh-session test	Same, real feature scale
Implementation	Guardrailed build + mean review	Same, plus full CI/CD

Lecture activities skip the heavier session-logging overhead and EGM process – that lives in the graded projects, starting with Project 2. This specific lecture does grade EGM process as this is what is taught.

Activity — Book Review: Truncating Review Text

Kicks off the **Book Review arc** — one app, growing one layer per lecture through Lecture 9. Today: a FastAPI backend with a working `GET /books` endpoint over mock data.

Your task: `truncate_review_text(text, max_length)` in `app/utils/text_format.py` — end a long review preview in an ellipsis without cutting a word in half.

Graded (10 pts): 6 tests passing · 2 EGM artifacts exist (context hierarchy + design doc) · 2 design-doc quality — not just "does the feature work."

Activity — In Class (Phase 1)

No Code Rule in effect — design only. Enter Plan Mode (`Shift+Tab` or `/plan`) so it's enforced by the tool, not just discipline:

1. Run `/design-discussion` — scopes the feature, loads the right `CLAUDE.md` context
2. Design-discuss in prose; apply the sycophant challenge
3. Run `/write-spec` to capture the design (`Ctrl+G` opens the plan to edit directly)

Activity — Before Next Lecture

(Phases 3–4)

Goldfish Protocol: fresh session, run `/goldfish-test` — fed only the spec + context hierarchy — comprehension test, critic review, revise the spec for anything it exposes.

Implementation: implement to the revised spec. `pytest tests/ -v` from `backend/` — all 12 tests should pass. Run `/mean-review` on the finished function.

Push to GitLab.

 **Project 1 due Tue Sep 15** (today)



Questions?

Before Lecture 5 — REST API Design & FastAPI Fundamentals read:

- [MDN: HTTP Methods](#)
- [FastAPI Tutorial](#)